



④ MULTITHREADED LINUX APPLICATION USING POSIX THREADS & MUTEXES

Parallel task execution with safe, race-free access to shared data on a multi-core system.

Presented by: Bala Pranitha 2520030051

S. Manvi Vaishnavi 2520030326

Rohini Tanmayi 2520030349

Guide: Bheekya

Department / Year: CSE 2025 - 2029

Operating Systems Project

TABLE OF CONTENTS

01 Introduction

02 Abstract

03 Objectives

04 Problem Statement

05 Proposed Workflow

06 Tools & Technologies

07 Expected Outcomes

08 Conclusion

INTRODUCTION



Idle Cores by Default

A single-threaded program runs on one core. Modern processors ship with 4 to 16 cores, so most of the available compute simply goes unused.



Threads Are Cheap on Linux

POSIX threads share one address space, which makes communication fast, and the pthreads API is portable across every UNIX-like system.



Shared Memory Is Risky

That same shared address space lets two threads update one variable at once. The result is wrong, and it fails silently.



DESIGN MODEL

Producer-consumer: one producer fills a bounded queue that N worker threads drain concurrently, merging partial results at the end.



KEY CONSTRUCTS

`pthread_create`, `pthread_join`, `pthread_mutex_lock`, `pthread_cond_wait` and `pthread_cond_signal` form the whole synchronisation surface.

ABSTRACT

Project Abstract

This project implements a multithreaded application on Linux using the POSIX threads library. A main thread partitions a large dataset into work items and places them in a bounded shared buffer, while a pool of worker threads removes and processes those items in parallel on separate cores.

Access to the buffer and to the aggregated results is serialised with pthread mutexes, and condition variables signal the not-full and not-empty states so that threads block rather than spin. All threads are joined cleanly and every mutex and condition variable is destroyed before exit. The implementation is validated for race-freedom with Valgrind's Helgrind and benchmarked against a sequential baseline to quantify the speed-up on a multi-core processor.

OBJECTIVES

01

IMPLEMENT

Build a multithreaded application in C on Linux using the pthreads API, and split a large dataset into work items served to a fixed pool of worker threads.

02

SYNCHRONISE

Guard every shared structure with `pthread_mutex_t`, and use condition variables for the not-empty and not-full states so idle threads block instead of busy-waiting.

03

VALIDATE

Prove race-freedom with Helgrind and ThreadSanitizer, then benchmark run time, speed-up and efficiency against a single-threaded baseline.



RACE CONDITIONS

Unguarded updates to shared counters and buffers interleave, so increments are lost and structures are read half-written. The failures are non-deterministic.



DEADLOCK & WASTE

Careless locking swings the other way: threads block on each other forever, or burn CPU in busy-wait loops while the queue is empty.

PROBLEM

Problem Statement

A workload large enough to need parallel execution must still produce exactly the same result as the sequential version. Without explicit synchronisation, the output of a multithreaded program depends on the scheduler, so it is neither correct nor reproducible.

The project must therefore guarantee mutual exclusion around every critical section, a consistent lock ordering, and blocking waits instead of spin loops, while still delivering a measurable speed-up over the single-threaded baseline.

PROPOSED WORKFLOW

01 DESIGN

Identify the parallelizable workload, fix task granularity, and list every shared variable.

02 SETUP

Ubuntu with GCC, a Make file that links `-pthread`, and a Git repository for version control.

03 BASELINE

Write and time the single-threaded version; it is the reference for correctness and speed.

04 THREADING

Add `pthread_create` and `pthread_join`, the worker routine and per-thread argument structs.

05 SYNCHRONISE

Introduce mutexes around critical sections and condition variables for queue signalling.

06 TESTING

Run Helgrind, vary the thread count, record timings and document speed-up and efficiency.

Phases 04 and 05 iterate: every new shared structure is re-analysed for critical sections before the next benchmark run.

TOOLS & TECHNOLOGIES

01 Ubuntu Linux 22.04 LTS

02 C (C11) compiled with GCC

03 POSIX threads, linked -pthread

04 Mutexes & condition variables

05 GDB, thread-aware breakpoints

06 Valgrind Helgrind, ThreadSanitizer

07 perf and clock_gettime timing

08 GNU Make and Git

EXPECTED OUTCOMES

3.5×

SPEED-UP

Projected run time on four cores falls from about 40 s to 11 s against the sequential baseline, with efficiency reported alongside.

0

DATA RACES

Helgrind and ThreadSanitizer report no data races and no lock-order violations across repeated runs at every thread count.

85%

CPU USAGE

Aggregate utilisation stays high because workers block on condition variables instead of spinning while the queue is empty.

Deliverables: documented source code and Makefile · Helgrind verification report · benchmark tables · project report. Figures are targets, to be confirmed during benchmarking.

 THANK YOU

CONCLUSION

The project demonstrates that concurrency and correctness are achievable together on Linux. POSIX mutexes and condition variables are sufficient to keep a producer–consumer workload race-free, while the worker pool delivers a measurable speed-up over the sequential baseline.

Correctness is established by tooling rather than by observation — a race that never fired is still a race.

Future work: reader–writer locks, a work-stealing pool, lock-free queues, and NUMA-aware thread placement.

Outcome

3.5× faster on 4 cores

Validation

Zero races under Helgrind

Future Scope

Lock-free queues